Programming Language 과제 2

Chapter 4_ Problem Set ~~1, 2, 3, 4, 5, 7~~

1. Perform the pairwise disjointness test (쌍방향 공통 테스트)

A → aB | b | cBB
B → aB | bA | aBb
C → aaA | b | caB

a. A → aB | b | cBB
First(aB) = a
First(b) = b
First(cBB) = c
→ They don't intersect and pass the test.
(최초의 터미널, 겹치는 원소 X)

b. B → aB | bA | aBb
First(aB) = a
First(bA) = b
First(aBb) = a
→ They intersect and fail the test.

c. C → aaA | b | caB
First(aaA) = a
First(b) = b
First(caB) = c
→ They don't intersect and pass the test.

2. Perform the pairwise disjointness test

S → aSb | bAA
A → b{aB} | a
B → aB | a

a. S → aSb | bAA
First(aSb) = a
First(bAA) = b
→ They don't intersect and pass the test.

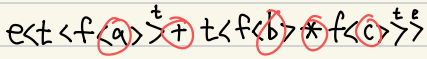
b. A → b{aB} | a
First(b{aB}) = b
First(a) = a
→ They don't intersect and pass the test.

c. B → aB | a
First(aB) = a
First(a) = a
→ They intersect and fail the test.

교재에 따르면 (INT_LIT (정수 리터럴) 10, IDENT (식별자) 11, ASSIGN_OP (할당 연산자) 20, ADD_OP 21, SUB_OP 22, MULT_OP 23, DIV_OP 24, LEFT_PAREN (() 25, RIGHT_PAREN ()) 26

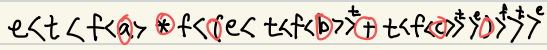
3. Show a trace of the recursive descent parser → a + b * c

Next token is: 11 Next lexeme is a
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 21 Next lexeme is +
Exit <factor>
Exit <term>
Next token is: 11 Next lexeme is b
Enter <term>
Enter <factor>
Next token is: 23 Next lexeme is *
Exit <factor>
Next token is: 11 Next lexeme is c
Enter <factor>
Next token is: -1 Next lexeme is EOF
Exit <factor>
Exit <term>
Exit <expr>



4. Show a trace of the recursive descent parser → a * (b+c)

Next token is: 11 Next lexeme is a
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 23 Next lexeme is *
Exit <factor>
Next token is: 25 Next lexeme is (
Enter <factor>



e<t<f<a> * f<(e< t<f<b>> + t<f<c>>)>>>

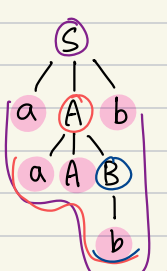
Next token is : 11 Next lexeme is b
Enter <expr>
Enter <term>
Enter <factor>
Next token is : 21 Next lexeme is +
Exit <factor>
Exit <term>
Next token is : 11 Next lexeme is c
Enter <term>
Enter <factor>
Next token is : 26 Next lexeme is)
Exit <factor>
Exit <term>
Exit <expr>
Next token is : -1 Next lexeme is EOF
Exit <factor>
Exit <term>
Exit <expr>

5. Draw a parse tree & show the parses and simple phrases, handle

S → aAb | bBA A → ab | aAB B → aB | b

a. aaAbb

S → aAb
A → aAB
B → b



? aaAbb → aaABb → aAb

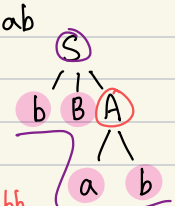
올바른 규칙의 유형이지만 사용할 수 없는 구문

Phrases : aaAbb, aAb, b

terminal로만 된 구문 → Simple phrases: b

last → Handle : b

b. bBab



Phrases : bBab, ab → ?

Simple phrases: ab

Handle : ab

c. aaAbBb

aaAbBb = aSBb = aSBB = X

↳ The last string cannot be derived from the given grammar.

? bBab → bBA

7. Show a complete parse (parse stack contents, input string, action for the string id * (id+id))

Stack	Input	Action
0	id * (id+id)$	Shift 5
0id5	* (id+id) $	Reduce 6 (Use GOTO [0,F])
0F3	* (id+id) $	Reduce 4 (Use GOTO [0,T])
0T2	* (id+id) $	Shift 7
0T2*7	(id+id) $	Shift 4
0T2*7(4	id+id) $	Shift 5
0T2*7(4id5	+id) $	Reduce 6 (Use GOTO [4,F])
0T2*7(4F3	+id) $	Reduce 4 (Use GOTO[4,T])
0T2*7(4T2	+id) $	Reduce 2 (Use GOTO[4,E])
0T2*7(4E8	+id) $	Shift 6
0T2*7(4E8+6	id)$	Shift 5
0T2*7(4E8+6id5	)$	Reduce 6 (Use GOTO[6,F])
0T2*7(4E8+6F3	)$	Reduce 4 (Use GOTO[6,T])
0T2*7(4E8+6T9	)$	Reduce 1 (Use GOTO[6,E])
0T2*7(4E8	)$	Shift 11
0T2*7(4E8)11	$	Reduce 5 (Use GOTO [7,F])
0T2*7F10	$	Reduce 3 (Use GOTO[0,T])
0T2	$	Reduce 2 (Use GOTO[0,E])
0E1	$	accept